

Furiosa RNGD 모델 준비

SDK 내부 구조와 빌드 에러 분석

furiosa-llm build 의 내부 동작 (검증 · preset · 버킷 · 컴파일) 을 SDK 소스로 직접 확인하고 ,
모델을 NPU 아티팩트로 준비하며 실제로 마주친 빌드 에러를 정리합니다 .

다룰 내용

빌드 3 단계 흐름 · tp/pp 규칙 · preset 매칭 원리와 함정 · 버킷 시스템 · 빌드 2 단계 (트레이싱 / 컴파일) · 빌드 에러 4 종 · 성공 · 실패 매트릭스

furiosa-llm build 한 줄이 하는 일

명령 하나 안에서 두 성격의 작업이 순서대로 — 입구 검증 / 채움 (가벼움) 과 실제 빌드 (무거움)

1. 준비

`__init__` 단계 (수 초)

- ① Validate — 설정 · HF config · 버킷이 규칙에 맞는지 검사
- ② Resolve — 안 준 값 (`max_model_len` · 버킷) 을 `config.preset` 에서 채움

2. 빌드

`build` 단계 (수 시간, 메모리 위험)

- ③ Tracing — 버킷마다 FX 그래프 생성 (`weight+IR` 메모리 ★)
- ④ Compile — supertask 마다 NPU 컴파일러로 EDF 바이너리

3. 저장

결과물

- `artifact.json` (메타 · `parallel_config`)
- `binary_bundle.zip` (EDF) · `config` · `tokenizer`

OOM · 실패는 거의 전부 ③ Tracing 단계 — ④ Compile("Compilation Progress") 로그가 뜨면 메모리 위험 구간은 통과한 것

tp / pp 규칙 — 입구에서 막는다

validate_parallel_config 가 비싼 빌드 전에 잘못된 병렬 설정을 즉시 거부

검증 규칙

- $\text{tensor_parallel}(tp) \in \{4, 8, 32\}$ — 그 외 값은 거부
- $\text{pipeline_parallel}(pp) \geq 1$
- 필요 디바이스 = $\text{ceil}(tp/8) \times pp \leq 8$
- RNGD 1 장 = 8 PE (NUM_PES_PER_NPU)
- $tp=4 \rightarrow$ 1 장 안 PE 4 개만 / $tp=8 \rightarrow$ 1 장 풀

tp	pp	카드 수	비고
4	1~8	1~8	PE 4 개만
8	1~8	1~8	1 장 풀 PE
32	1~2	4~8	4 장에 분산

prebuilt 32B·70B 아티팩트가 $tp=32$ — 그래서 RNGD 4 장 (32 PE) 이 있어야 서빙됨
(2 장에선 "Required PEs: 32" 거부)

버킷 자동 선택 — find_preset

버킷을 직접 안 주면 모델에 맞는 preset 을 SDK 가 골라줌

2 단계 매칭

1 단계 model_type 으로 후보 필터 (llama·qwen2·qwen3·qwen3_moe·exaone4 ...)

2 단계 (hidden_size, intermediate_size) 로 layer 당 파라미터 수 계산 → log-distance 가 가장 가까운 preset 선택

- 크기가 정확히 같지 않아도 됨 — 가장 가까운 항목이 잡힘 (같은 architecture 의 fine-tune 모델은 대체로 자동 매칭)
- 예 : qwen3_moe(2048,6144)=Qwen3-Coder-30B vs (6144,8192)=Qwen3-Coder-480B → per-layer 0.042B vs 0.252B, log-distance 로 명확히 구분
- preset 이 아예 없는 model_type(mistral·phi3·gpt_oss 등) 은 -pb/-db 로 버킷을 직접 줘야 빌드 가능

합정 — (h,i) 가 같으면 " 등록 순서 " 가 결정한다

log-distance 가 동점일 때 min() 은 먼저 등록된 항목을 고름

```
PRESET_REFS — qwen3_moe (2048, 6144) 3 개 중복  
  
# 먼저 등록된 일반 preset (decode 8192)  
QWEN_3_30B_A3B_PRESET ← 먼저 → 선택됨  
QWEN_3_CODER_30B_A3B_PRESET (decode 256K)  
QWEN_3_CODER_30B_A3B_PRESET ( 중복 )  
  
→ Qwen3-Coder 가 8192 preset 에 매칭됨
```

증상

- Qwen3-Coder-30B(256K 모델) 이 컨텍스트 8192 로만 빌드됨
- 두 preset 의 (h,i) 가 완전히 동일 → log-distance=0 동점
- 해결: 일반 entry 제거 → Coder preset(256K) 로 통일

교훈 — preset 을 새로 추가할 때 기존과 (hidden, intermediate) 가 겹치면, 의도한 preset 을 PRESET_REFS 에서 먼저 등록해야 한다

버킷 — AOT 컴파일이라 모양을 미리 굽는다

(batch, context) 조합 하나하나를 그래프로 컴파일해 두고, 요청을 가장 가까운 버킷에 라우팅

prefill

`kv_cache = 0`

프롬프트 전체를 한 번에 — 첫 토큰
직전. 보통 짧게 (128~1024)

decode

`input_ids = 1`

한 토큰씩 생성. 생성 모델 필수 —
최대 컨텍스트를 여기서 정함

append

`1<input<attn`

prefix 캐시 일부 재사용 + 새 토큰
묶음 추가 (CLI 없음, API)

tokenwise

정수 1 개

composable kernel 용 시퀀스
길이 (CLI 없음, API)

모든 요청 모양을 다 구울 필요 없음 — 대표 모양만 빌드. `--max-model-len` 으로 큰 버킷을 제외해 빌드 메모리를 줄일 수 있음

Tracing vs Compile — 성격이 정반대

같은 build 명령 안의 두 단계가 메모리 · 시간 특성이 완전히 다름

③ Tracing

"Model Tracing Progress: x/Y"

- 버킷마다 PyTorch FX 로 그래프 생성
- 전체 weight + 모든 버킷 IR 을 한 메모리에 누적
- ★ OOM 위험 매우 높음 (32B 는 100GB+ 도달)
- 시간 비중 ~1/4

④ Compile

"Compilation Progress: x/Y"

- supertask(보통 block 1 개) 마다 NPU 컴파일 → EDF
- 현재 supertask 만 메모리에 — 누적 거의 없음
- 메모리 안정 (32B 실측 ~20GB)
- 시간 비중 ~3/4 (오래 걸리지만 안전)

실측: 32B 빌드 OOM 4 회 모두 Tracing 단계 — "Compilation Progress" 가 한 번 뜨면 그 뒤로는 시간만 기다리면 됨

에러 ① — BF16 32B 를 tp=32 로 빌드

Tracing 은 통과하지만 Compile 진입 직후 컴파일러가 거부

Compile 단계 에러

```
RuntimeError: Compilation error: fail to compile:  
Graph input#0 must have Broadcast or Fixed  
DramShapeGuide (Name: embedding_table)
```

- 32B dense 모델의 BF16 임베딩을 tp=32 로 분할할 때 컴파일러가 DRAM shape guide 를 만들지 못함 — SDK 한계
- 재현 : Qwen3-32B(BF16,tp32) · Qwen2.5-Coder-32B(BF16,tp32) 둘 다 동일
- 회피 : 같은 모델의 FP8 변종 사용 — 양자화 메타가 임베딩에 함께 들어가 컴파일 가능 (FP8 빌드는 성공)

에러 ② — FP8 을 tp=32 로 분할 (block 정렬 깨짐)

FP8 양자화 블록 (128×128) 이 tp 분할 후 차원과 안 맞음

```
Compile stage_2 에러  
  
RuntimeError: Compilation error:  
impossible to apply  
reshape mapping
```

hidden 2048 ÷ tp 32 = 64

< FP8 block_size 128 → 정렬 불가

tp	hidden ÷ tp	block(128) 정렬	결과
8	256	≥ 128 OK	성공
16	128	= 128 경계	이론상 가능
32	64	< 128 깨짐	실패

hidden 이 작은 MoE 모델 (Qwen3-Coder-30B, hidden 2048) 은 tp=32 에서 FP8 블록 정렬이 깨짐 — tp=8 로는 정상 빌드

에러 ③ — OOM, swap 이 있어도 못 막는다

Tracing 의 working set 이 RAM 을 넘으면 커널 OOM killer 가 발동

```
journalctl
```

```
tmux-spawn-...scope: A process of this unit has been  
killed by the OOM killer.
```

왜 swap 이 안 통하나

- Tracing 메모리는 계속 쓰이는 active working set
- active 페이지는 swap 으로 못 빠짐
- 실측 : swap 207GB 중 1.5GB 만 쓰고 OOM

진짜 해결책

- `--max-model-len` 축소 → working set 자체를 줄임
- FP8 로 weight 절반 (60GB→30GB)
- systemd-oomd 꺼도 커널 OOM 은 별개 (못 끄م)

에러 ④ — 그 외 자주 막히는 것들

컴파일 이전 단계에서 빌드를 멈추는 환경 · 모델 제약

디스크 풀

No space left on device → 빌드가 import 단계에서 즉시 종료 (임시 디렉토리 못 만들). 앞 모델 다운로드가 디스크를 채운 게 원인

vocab_size 패딩

SDK 가 vocab 을 64 의 배수로 패딩 (예 32016→32064) 하는데 weight loader 가 패딩을 못 받아 mismatch — CodeLlama 빌드 불가

SSH 끊김

빌드는 수 시간 — 직접 셸이면 연결이 끊길 때 같이 죽음 . tmux 세션 안에서 빌드해야 살아남음

pp 미지원 architecture

-pp 는 block_slicer.py 에 등록된 4 종 (Llama·GPTJ·Bert·Roberta) 만 가능 . 그 외는 NotImplementedError → dp 로 우회

빌드 성공 · 실패 매트릭스

같은 모델도 dtype·tp 조합에 따라 결과가 갈림

모델	dtype	tp	결과	원인 / 비고
Qwen2.5-Coder-1.5B/7B/14B	BF16	8	성공	1 장 적재 (max 32K)
Qwen3-32B-FP8	FP8	8	성공	1 장 적재 (max 40960)
Qwen3-Coder-30B-A3B-FP8	FP8	8	성공	MoE, 1 장 적재
Qwen3-32B / Qwen2.5-Coder-32B	BF16	32	실패	embedding_table (에러①)
Qwen3-Coder-30B-A3B-FP8	FP8	32	실패	reshape mapping (에러②)
EXAONE-4.0-32B	BF16	32	실패	OOM + 디스크 풀 (에러③④)
CodeLlama-70B	BF16	32	실패	vocab 패딩 mismatch (에러④)

일관된 패턴 : FP8 + 작은 tp(8) 는 성공 / BF16 32B + tp=32 는 컴파일 미지원

정리 — 모델 준비 체크리스트

성공 경로

FP8 양자화 + tp=8 → 1 장 적재 + Tracing 메모리 절반 . 32B 급은 이 조합이 표준

BF16 32B tp=32 회피

embedding_table 컴파일 미지원 — 메모리를 늘려도 안 됨 . FP8 변종으로 전환

preset 순서 주의

(hidden, intermediate) 가 겹치는 preset 은 의도한 것을 먼저 등록 (동점 시 first 선택)

OOM 은 메모리 축소로

swap·oomd 끄기로는 못 막음 . --max-model-len 을 줄여 working set 자체를 낮춰야

" 빌드 가능 " ≠ " 우리 머신서 서빙 가능 " — 빌드는 host RAM, 서빙은 NPU HBM. 둘을 따로 확인해야 한다